(even though they have the same attributes) than another set of nested user elements created by another iteration of `d3.nest`.

This is easier to understand in practice. The code in listing 9.4 shows two uses of `d3.nest` to create the same nested structure with the same data. But when you check to see if one element in the first array is the same as an element in the second array, you see that they're different objects, even though they look the same.

Listing 9.4 Comparing nested data

```
var nestedTweets = d3.nest()
  .key(function (el) {return el.user})
  .entries(incData);

var nestedTweets2 = d3.nest()
  .key(function (el) {return el.user})
  .entries(incData);

packableTweets = {id: "root", values: nestedTweets};

nestedTweets.value[0] == nestedTweets2.value[0]
nestedTweets[0] == nestedTweets2[0]
packableTweets.values[0] == nestedTweets[0]
packableTweets.values[0] == nestedTweets2[0]
```

Seems to be a duplicate of nestedTweets but is a different object

True

False

True

False

Keeping this in mind, we create the chart function in listing 9.3 to pass datasets that have shared elements. Because the objects are shared, we can identify which graphics (whether a circle or rectangle or line of a spreadsheet) refer to the same object.

9.1.2 Spreadsheet

The first chart we'll create is the same spreadsheet we made in chapter 8, except without any of the sorting buttons. It takes the object array as it appears in tweets.json, which we can pass straight to our `createSpreadsheet` function in the following listing. I'm not going to dwell on the individual charting functions like this, which are explained in more detail in earlier chapters.

Listing 9.5 The spreadsheet code

```
function createSpreadsheet(incData, targetDiv) {

  var keyValues = d3.keys(incData[0]);

  d3.select(targetDiv)
  .append("div")
  .attr("class", "table");

  d3.select("div.table")
  .append("div")
  .attr("class", "head row")
  .selectAll("div.data")
  .data(keyValues)
  .enter()
  .append("div")
```